

CNuvS Ex. 11 - Jan Lindauer, Paul Feidieker

Task 4

- (a) The first distinct operational risk is the missing redundancy. If for some reason the supercomputer in Frankfurt goes offline (power outage, crashes, cyberattack etc.), no one can access websites using standard URLs anymore, since there's no other way to translate domain names.
- The second operational risk is latency and traffic. Users sitting far away from Frankfurt would suffer from massive delays just because of the distance to the server. Another problem would be the combined traffic of the whole world, choking Frankfurts network.

The solution to the redundancy issue is the hierarchical, decentralized structure of DNS that is backed by Anycast routing. The 13 global root servers are duplicated into thousands of instances. If one instance goes out, traffic is just rerouted to the next closest one.

The solution to the latency and traffic issue is the caching logic of DNS. Previously visited IPs are saved locally for some time (TTL) and only a small amount of queries creates global traffic.

[1, p. 21, 24]

- (b) In an iterative request query a server replies with the name of the server to contact if itself can't resolve the requested domain.
- In a recursive request query the burden of name resolution lies on the contacted name server.
- [1, p. 26, 27]
- (c) The mechanism causing this is DNS caching. Users that have a valid Resource Record will access the old website until the TTL is over. This helps performance since the user doesn't have to request a domain resolve everytime which could take a while depending on where the DNS-Server is located. The delay could be resolved by lowering the TTL to for example 5 minutes. Then wait for the duration of the old TTL. After this the updated IP would propagate worldwide within 5 minutes.
- [1, p. 24]

Task 5

- (a) HTTP/1.1 requires requests on a single connection to be answered in the exact order they were sent. If the first request is slow, all subsequent requests are blocked and must wait in line.
- HTTP/2 splits requests and responses into small frames and interleaves them over a single TCP connection. A slow image no longer blocks a fast stylesheet.

HTTP/3 replaces TCP with QUIC. QUIC handles streams natively at the transport layer. If a packet for one stream is lost, only that specific stream pauses. All other streams continue downloading without interruption.

[2], [3]

- (b) **Session creation:**
- When a user visits the site, the browser sends an initial stateless HTTP request. Because no session exists yet, the server recognizes this as a new visitor. The server creates a unique Session ID and initializes a corresponding data record.

Setting the Cookie:

To establish the session, the server sends an HTTP response containing the Set-Cookie header line, which passes the Session ID to the browser.

Example: "Set-Cookie: session_id=xyz; Secure; HttpOnly"

Maintaining the session:

For every sequential click or request after that, the browser automatically reads its cookie store and includes that same Session ID in the Cookie header line of the HTTP request.

Example: "Cookie: session_id=xyz"

Server-Side Lookup:

When the server receives subsequent requests, it reads the Cookie header, extracts the Session ID, and uses it as a key to look up and update that specific user's stateful data record.

The actual data record is stored securely on the server side, while the browser only holds the Session ID.

[1, p. 49, 50]

- (c) "HTTP 429 Too Many Requests" indicates that the client has exceeded the server's rate limits. The server should include the Retry-After header to tell the client how long to wait before trying again.
[4], [5]

Sources

- [1] Prof. Dr. Marco Zimmerling, “CNuvS 07: Application Layer.” 2026.
- [2] Cloudflare, [Online]. Available: <https://www.cloudflare.com/learning/performance/what-is-http3/>
- [3] MDN, [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/HTTP/Basics_of_HTTP/Evolution_of_HTTP
- [4] MDN, [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Status/429>
- [5] MDN, [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Retry-After>